
ECE 592 (109) HW2

— Deadline: 27 Sept 2026 —

Outline

- Task Introduction
- Dataset
- Evaluation Metrics
- Submission
- Grading Policy
- Baseline Hints

Task Introduction - Image Classification

- Solve image classification with convolutional neural networks (CNN).



Dataset

- The images are collected from the food-11 dataset splitted into 11 classes.
- Training set: 10000 labeled images
- Validation set: 3643 labeled images
- Testing set: 3000 images without labeled

Evaluation Metrics

- Accuracy on testing set

$$\text{Acc} = \frac{\text{pred} == \text{label}}{\text{len}(\text{data})} \cdot 100\%$$

Q1. Augmentation Implementation

Implement augmentation by finishing **train_tfm** in the code with image size of your choice. Explain the effects of transformations in your report.

- Your **train_tfm** must be capable of producing 5+ different results when given an identical image multiple times.
- Your **train_tfm** in the report can be different from train_tfm in your training code.

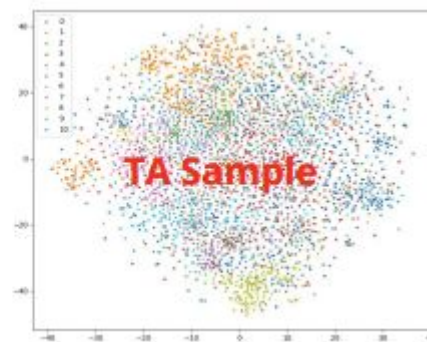
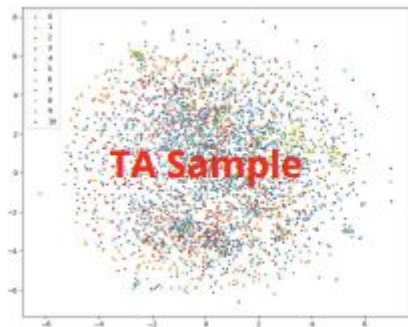


X

O

Q2. Visual Representations Implementation

- Visualize the learned visual representations of the CNN model on the **validation set** by implementing t-SNE (t-distributed Stochastic Neighbor Embedding) on the output of both **top & mid** layers (You need to submit **2** images).
- Briefly explain your result of the t-SNE visualization.



Q2. Visual Representations Implementation

- Take the CNN architecture beside as an example.
- The layers contained within the red box may be identified as the "Bottom layers."
- The layers contained within the green box may be identified as the "Mid layers."
- The layers contained within the blue box may be identified as the "Top layers."

```
self.cnn = nn.Sequential(  
    nn.Conv2d(3, 64, 3, 1, 1), # [64, 128, 128]  
    nn.BatchNorm2d(64),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2, 0),      # [64, 64, 64]  
  
    nn.Conv2d(64, 128, 3, 1, 1), # [128, 64, 64]  
    nn.BatchNorm2d(128),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2, 0),      # [128, 32, 32]  
  
    nn.Conv2d(128, 256, 3, 1, 1), # [256, 32, 32]  
    nn.BatchNorm2d(256),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2, 0),      # [256, 16, 16]  
  
    nn.Conv2d(256, 512, 3, 1, 1), # [512, 16, 16]  
    nn.BatchNorm2d(512),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2, 0),      # [512, 8, 8]  
  
    nn.Conv2d(512, 512, 3, 1, 1), # [512, 8, 8]  
    nn.BatchNorm2d(512),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2, 0),      # [512, 4, 4]  
)
```


Submission

1. Submit the jupyter notebook file which contains the running results, and a report explaining the results of question 1 and 2.
2. Include the final accuracy in the report.
3. Submission Deadline: **27 Sept 2026, Sunday**

Grading Policy - Baseline (1/2)

Baseline	Accuracy	Hints	Training Time (Reference)
Simple	0.637	Run Sample Code	0.5hr-1hr on Colab
Medium	0.700	Do some Data Augmentation and Train longer	1.5hr - 2hr on Colab
Strong	0.814	Use predefined CNN from torchvision or tensorflow	10hr - 12hr on Colab (Using Kaggle is suggested)
Boss	0.874	Cross Validation + Ensemble or any other methods you know	20hr+ on Kaggle

Grading Policy - HW Score (2/2)

- simple (public) +0.5 pt
- simple (private) +0.5 pts
- medium (public) +0.5 pts
- medium (private) +0.5 pts
- strong (public) +0.5 pts
- strong (private) +0.5 pts
- boss (public) +0.5 pts
- boss (private) +0.5 pts
- code submission +2 pts
- report +4 pts

Total : 10 pts

Hints for Baseline

Data Augmentation

- Modify the image data so non-identical inputs are given to the model each epoch, to prevent overfitting of the model
- Visit [torchvision.transforms](#) for a list of choices and their corresponding effect. Diversity is encouraged! Usually, stacking multiple transformations leads to better results.
- Coding : fill in **train_tfm** to gain this effect



Model Selection

- Visit [torchvision.models](https://pytorch.org/vision/models) for a list of model structures, or go to [timm](https://github.com/rwight/pytorch-image-models) for the latest model structures.
- Pretrained weights are not allowed.
 - Torchvision before 0.13 -> `pretrained=False`
 - After 0.13 -> `weights=False`

Classification

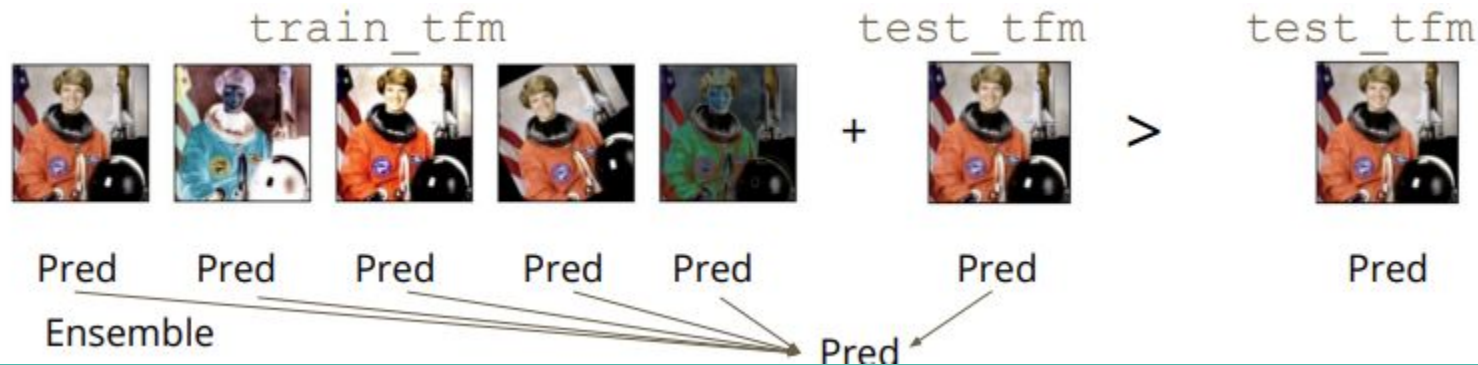
The models subpackage contains definitions for the following model architectures for image classification:

- AlexNet
- VGG
- ResNet
- SqueezeNet

Pre-defined model

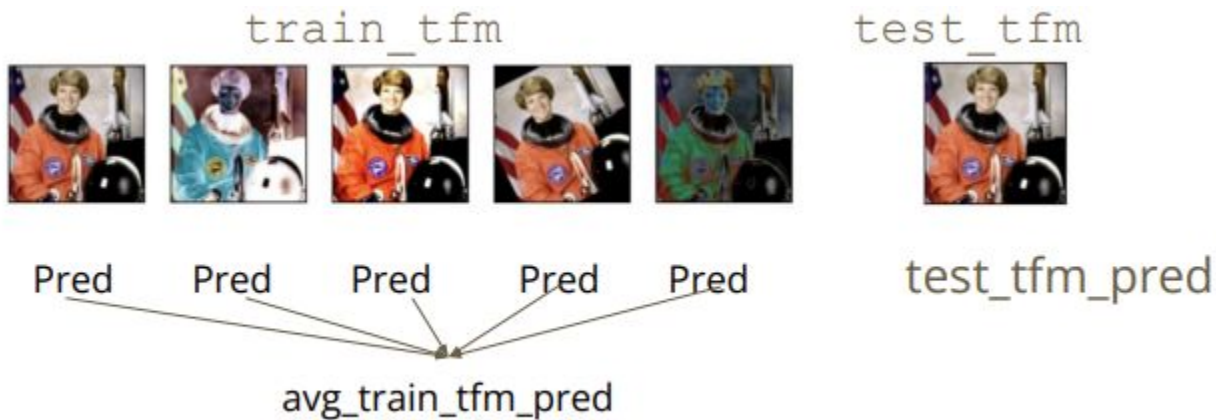
Test Time Augmentation

- The sample code tests images using a deterministic “test transformation”
- You may use the train transformation for a more diversified representation of the images, and predict with multiple variants of the test images.
- Coding : You need to fill in **train_tfm**, change the augmentation method for test_dataset, and modify prediction code to gain this effect



Test Time Augmentation

- Usually, test_tfm will produce images that are more identifiable, so you can assign a larger weight to test_tfm results for better performance.
- Ex : Final Prediction = avg_train_tfm_pred * **0.2** + test_tfm_pred* **0.8**

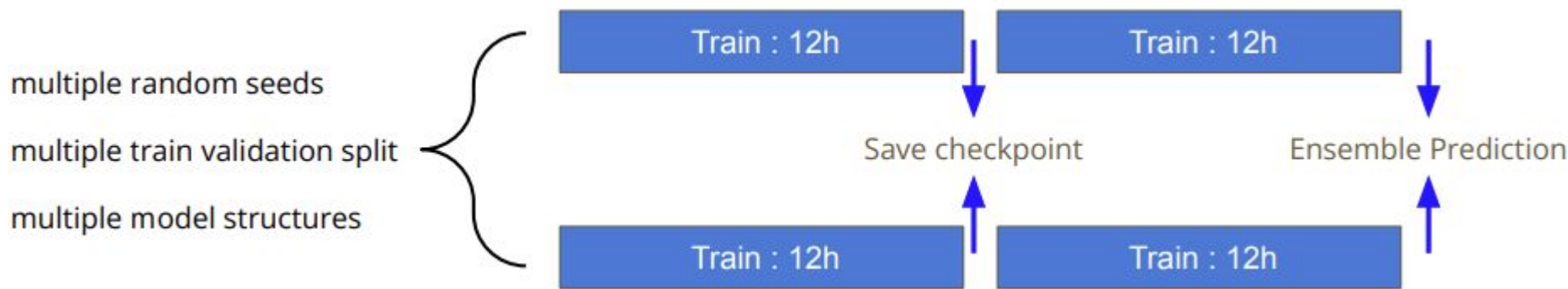


A sample procedure for beating the boss baseline

- The boss baseline might not be beaten with a single model.



- Your procedure can be ensemble of multiple models with parallelization



Cross Validation

- Cross-validation is a resampling method that uses different portions of the data to validate and train a model on different iterations. Ensembling multiple results leads to better performance.
- Coding : You need to merge the current train and validation paths, and resample from those to form new train and validation sets.



Ensemble

- Average of logits or probability : Need to save verbose output, less ambiguous
- Voting : Easier to implement
- Coding : basic math operations with numpy or torch

Experimental Tips

- Augmentation is a must to prevent overfitting. A good augmentation can carry on to the testing phase with Test Time Augmentation.
- If you build your own network structure and have implemented augmentation, don't be afraid to scale up your model. (Many predefined models structure are huge and perform great)
- Model structures with subsampling (max pooling) can work better, simply choosing the best performing models on ImageNet according to websites is not always a good idea because pretrained weights are not allowed.

Other Tricks

- On Classification
 - Label Smoothing Cross Entropy Loss
- On Optimization
 - Dropout
 - BatchNorm
 - Gradient Accumulation